Lambo: A Living Topological Memory Substrate for Multi-Agent Software Development

Narayan SS
nryn@lambo.dev

September 5, 2026

Abstract

Autonomous coding agents operate within ephemeral process lifecycles. When agents complete execution, their intermediate reasoning and architectural context vanish. Standard vector stores suffer from semantic drift and lack topological dependency awareness. We present Lambo, an in-memory topological memory daemon for multi-agent software engineering. Lambo models development state as a directed typed graph with an earned canonization lifecycle. Concurrent agents coordinate through an asynchronous write queue with monotonic receipts. Context retrieval uses three-leg hybrid candidate scoring and prioritized breadth-first topological expansion. Phase one merges lexical, recency, and dense vector signals. Phase two traverses structural dependency edges while respecting invalidation semantics. Phase three enforces canonical-first ranking and hot-list conflict preservation under token budgets. We evaluate Lambo across two production rigs using continuous 5-minute health sampling. This evaluation covers 2,494 heartbeat snapshots on CUDA and 3,429 on Metal over 12 to 16 days of deployment. Deduplication rates reached 12.0% during synchronized review swarms on Metal (against 0.9% in single-agent sessions) and 4.3% among swarm-named agents on CUDA (against 1.7% in non-swarm streams, 2.2% aggregate). Zero infrastructure-level faults occurred across all checkpoints. Tool-level execution misses (25% to 28% on inspect) revealed a text-surface affordance gap that motivates rendering explicit node identifiers. We formulate the multi-repository dispersion effect to explain why consensus promotion requires namespace scoping. Finally, we characterize hardware-specific memory paging overheads on unified memory platforms.

1 Introduction

Autonomous software engineering agents increasingly collaborate in swarms. Modern workflows deploy concurrent agents to refactor code, run tests, and remediate review findings. However, each agent invocation is fundamentally ephemeral. When an agent session terminates, its internal reasoning context disappears. Subsequent agents must re-derive architectural decisions from scratch.

Existing memory paradigms fail to maintain coherent state in software projects. Flat markdown scratchpads lack structural relationships. They cannot express multi-hop dependency chains or track invalidated decisions. Flat vector retrieval stores [1–3] evaluate pairwise cosine similarity between query text and isolated text chunks. In technical codebases, flat vector search suffers from semantic drift. It fails to distinguish between active constraints and superseded implementations. Static graph retrieval frameworks [4–6] require batch corpus indexing. They cannot ingest continuous mutations from concurrent agent processes.

To resolve these challenges, we present Lambo. Lambo is an in-memory topological memory daemon designed for multi-agent development. Lambo runs as a local supervised daemon exposing a Model Context Protocol (MCP) interface. Agents record typed architectural concepts and directed relationships during execution. Concepts represent logic invariants, structural constraints, and runtime observations.

Lambo coordinates concurrent agents without distributed locking. Mutations enter an asynchronous write queue that returns monotonic integer receipts within one millisecond. Agents verify write completion through receipt barriers, guaranteeing read-your-writes consistency. Retrieval operates via a three-phase hybrid topological pipeline. Phase one computes the maximum merge across keyword, recency, and dense vector candidates. Phase two performs prioritized breadth-first search across architectural dependency edges. Phase three applies daemon composite scoring, hot-list conflict preservation, and canonical-first ordering.

We evaluated Lambo continuously across discrete GPU and unified memory architectures. Our dataset covers continuous 5-minute daemon health sampling across 12 days on CUDA and 16 days on Metal. The telemetry confirms zero infrastructure failures, zero dropped ledger lines, and zero buffer saturations. We find that write-time deduplication correlates with concurrent task overlap across independent platforms. We identify contrasting verbal length signatures across frontier language models. We formalize why consensus promotion freezes under multi-repository development. Finally, we diagnose tool-level affordance errors and memory compression penalties

on host platforms.

2 System Architecture and Invariants

2.1 Single-Writer Daemon Topology

Figure 1 illustrates the system architecture of Lambo. Lambo enforces a single-writer daemon topology. One supervised background daemon holds exclusive write authority over a durable session. Agents communicate with the daemon via Unix domain sockets or local HTTP streams. This eliminates multi-process locking contention on persistent database engines. Simultaneously, read-only analytics query the shared memory graph concurrently without taking write locks.

2.2 Asynchronous Write Queue and Receipts

Memory writes must never block the reasoning loop of an agent. In contrast to operating system interrupt paging in MemGPT [7], Lambo uses an asynchronous ingestion model. All write operations enter a bounded asynchronous write queue. When an agent submits a mutation, the daemon immediately mints a monotonic integer receipt $r \in \mathbb{N}$. The daemon dispatches this receipt in sub-millisecond latency. The agent proceeds without awaiting vector embedding generation or disk serialization.

Unlike centralized policy brokers in MemClaw [8], Lambo enforces consistency through monotonic receipt barriers. An ingestion worker drains the queue in batches. Algorithm 1 specifies this ingestion process. The worker computes dense embeddings for novel concepts and commits graph mutations atomically. To enforce read-your-writes consistency, callers check receipt status with a bounded wait parameter. When subsequent operations depend on uncommitted writes, the agent waits on the specific receipt rung. Empirical production measurements demonstrate that this asynchronous receipt mechanism eliminates the need for advisory distributed locks.

2.3 Level-B Storage Pluggability

Lambo implements Level-B storage pluggability via compile-time Cargo feature flags. The core graph engine remains completely decoupled from the underlying storage backend. Supported engines include embedded SQLite, PostgreSQL, and CockroachDB. Every adapter satisfies a formal storage contract. This abstraction guarantees identical ranking outputs, topological consistency, and graph traversal logic across all backends.

2.4 The Embedding Contract Invariant

Vector spaces are fragile under encoder model migrations. Comparing vector embeddings generated by divergent models corrupts metric spaces. Lambo enforces an immutable session-level embedding contract. The contract is defined by a three-tuple:

$$\mathcal{C} = (\kappa, \mu, d) \tag{1}$$

Algorithm 1 Asynchronous Ingestion and Receipt Scheduling

Require: Queue $\mathcal{Q}_{\text{in}}$, In-Memory Graph $\mathcal{G}$, Persistent Store $\mathcal{S}$
Ensure: Monotonic execution and durable transaction commitments

```
function SubmitMutation(m, agent_id)
    r ← AtomicFetchAdd(NextReceipt, 1)
    Q_in.Push(Job(r, m, agent_id, Timestamp()))
    return r
end function

function IngestionWorkerLoop
    while DaemonIsRunning() do
        B ← Q_in.DrainBatch(MaxBatchSize)
        C_new ← {c ∈ m.Concepts | m ∈ B and c ∉ G}
        E_new ← BatchEmbed(C_new)
        for c ∈ C_new do
            c.embedding ← E_new[c]
        end for
        S.BeginTransaction()
        for job ∈ B do
            G.ApplyMutation(job.m)
            S.PersistMutation(job.m)
            UpdateDurableReceipt(job.r)
        end for
        S.CommitTransaction()
        NotifyWaiters(MaxDurableReceipt)
    end while
end function
```

Here, κ identifies the provider, μ defines the model name, and d specifies the exact vector dimensionality. The storage engine records $\mathcal{C}$ at session initialization. Any subsequent process that attaches with a mismatched contract is rejected immediately. This structural invariant guarantees mathematical purity across all vector search operations.

3 Formal Memory Model

3.1 The Directed Attributed Multi-Graph

Lambo models development state as a directed, attributed multi-graph:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \tag{2}$$

The node set $\mathcal{V}$ represents typed concepts. Each concept node $c \in \mathcal{V}$ possesses a type classification:

$$\tau(c) \in \{\text{Logic}, \text{Constraint}, \text{Observation}, \text{Entity}, \text{Resource}\} \tag{3}$$

Each node maintains a text payload, creation timestamp t_c, access counter α_c, and dense embedding $\mathbf{e}_c \in \mathbb{R}^d$.

Edges in $\mathcal{E}$ represent typed architectural dependencies:

$$\epsilon(e) \in \{\text{Dependency}, \text{Causal}, \text{Hierarchical}, \text{CoOccurrence}, \text{Semantic}\} \tag{4}$$

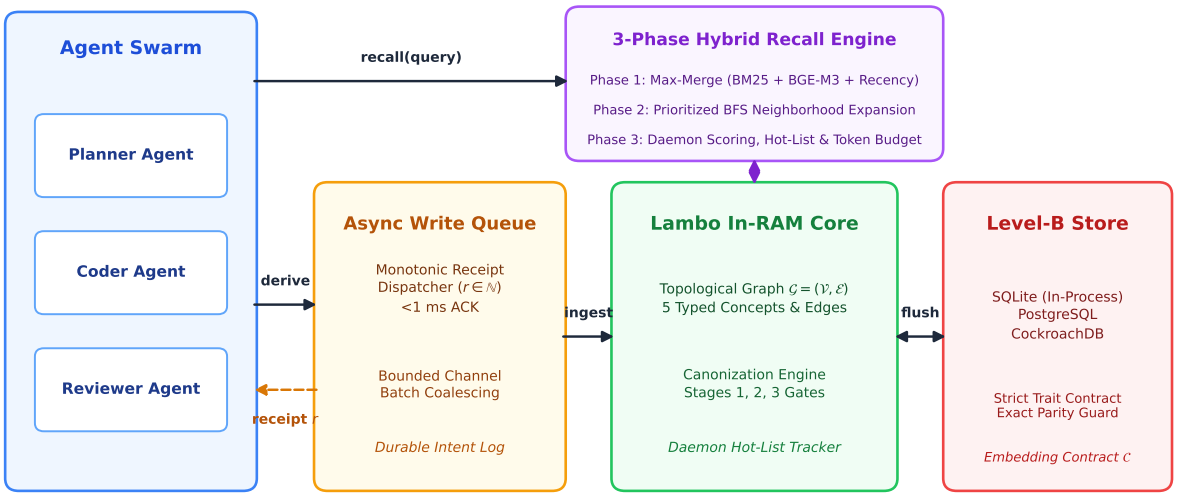

Figure 1: The Lambo system architecture. Concurrent agents submit asynchronous mutations and receive monotonic receipts. The background daemon updates the in-memory graph and flushes state to pluggable Level-B persistence engines.

In addition, bipartite provenance edges track agent actions. These include **Derives** edges from interactions to concepts, **Temporal** ordering edges, and explicit **Invalidates** edges.

3.2 Three-Leg Hybrid Candidate Retrieval

Given a natural language query q, the recall engine retrieves candidate nodes from three orthogonal legs. First, the lexical leg evaluates term frequencies over an inverted text index using BM25 [9].

$$S_{\text{lex}}(q, c) = \text{BM25}(q, c) \tag{5}$$

To prevent lexical candidates from being evicted by flat scores, the lexical search over-fetches with multiplier $M = 4$.

Second, the recency leg retrieves concepts derived within the $N_{\text{recent}} = 3$ most recent interactions. In contrast to the unbounded forgetting curve in MemoryBank [2] and linear recency in Generative Agents [10], Lambo enforces a calibrated floor. Members of this leg receive a flat floor score:

$$S_{\text{rec}}(c) = \rho_{\text{floor}} = 0.35 \tag{6}$$

Calibration against BGE-M3 text embeddings [11] establishes that genuine semantic matches score above 0.3991. Setting $\rho_{\text{floor}} = 0.35$ provides recency awareness without corrupting semantic ranking.

Third, the vector leg projects query text into vector $\mathbf{e}_q \in \mathbb{R}^d$ and computes cosine similarity.

$$S_{\text{vec}}(q, c) = \cos(\mathbf{e}_q, \mathbf{e}_c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_c\|_2} \tag{7}$$

Phase one merges candidate scores using a maximum selection rule.

$$S_1(q, c) = \max\left(S_{\text{lex}}(q, c), \quad S_{\text{vec}}(q, c), \quad S_{\text{rec}}(c)\right) \tag{8}$$

The merged candidate list is sorted in descending score order and truncated to limit K_1.

3.3 Phase-2 Prioritized Breadth-First Expansion

The candidate set forms the frontier for Phase-2 graph traversal. Unlike Personalized PageRank in HippoRAG [5, 6], Lambo prioritizes directed architectural edges. Starting from Phase-1 seeds at depth zero, Lambo executes a breadth-first search up to depth $D_{\max} = 2$. At each frontier node, traversal follows edges according to strict type priority:

$$\begin{aligned}\text{Dependency} \succ \text{Causal} \succ \text{Hierarchical} \\ \succ \text{CoOccurrence} \succ \text{Semantic}\end{aligned} \tag{9}$$

Provenance edges are excluded from Phase-2 expansion. A visited set cycle guard ensures that each concept is enqueued at most once. The earliest discovery path defines node membership.

Furthermore, Lambo applies a transitive closure over chunk grouping identifiers. Concepts sharing a `chunk_group_id` with an expanded node are force-included as siblings. Phase-1 candidates retain their S_1 scores, whereas newly expanded neighbors enter Phase 3 with an initial score of zero.

3.4 Phase-3 Composite Scoring and Token Assembly

Phase three computes final composite scores for all expanded concepts:

$$S_{\text{final}}(c) = w_{\text{daemon}} \cdot S_{\text{daemon}}(c) + w_{\text{query}} \cdot S_1(q, c) \quad (10)$$

Here, $w_{\text{daemon}} = 0.4$ and $w_{\text{query}} = 0.6$. The daemon score $S_{\text{daemon}}(c) \in [0, 1 + B_{\max}]$ evaluates session-relative topological health:

$$S_{\text{daemon}}(c) = 0.25 \cdot R(c) + 0.20 \cdot F(c) + 0.20 \cdot A(c) + 0.35 \cdot \Delta(c) + \beta_{\text{edge}}(c) + \mu_{\text{type}}(\tau(c)) \quad (11)$$

The normalized component metrics are defined as follows:

- **Recency:** $R(c) = \frac{t_{\text{touch}}(c) - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}}$, normalized over session duration.
- **Frequency:** $F(c) = \min\left(1.0, \frac{\alpha_c}{10.0}\right)$, saturating at ten accesses.
- **Session Activity:** $A(c) = \frac{|\{i \in \mathcal{I} | i \xrightarrow{\text{Derives}} c\}|}{|\mathcal{I}|}$.
- **Graph Density:** $\Delta(c) = \frac{\deg(c)}{\max_{u \in \mathcal{V}} \deg(u)}$, scaled by maximum degree.
- **Edge Bonus:** $\beta_{\text{edge}}(c) = \min\left(0.25, \sum_{e \sim c} b(\epsilon(e))\right)$.
- **Type Modifier:** $\mu_{\text{type}}(\text{Constraint}) = +0.15$, $\mu_{\text{type}}(\text{Entity}) = +0.05$, $\mu_{\text{type}}(\text{Logic}) = +0.05$, $\mu_{\text{type}}(\text{Resource}) = 0.0$, and $\mu_{\text{type}}(\text{Observation}) = -0.10$.

3.5 Canonical-First Ordering and Token Budgets

Hits are sorted by canonical priority and final score:

$$\text{RankKey}(c) = (\mathbb{I}_{\text{canonical}}(c), \quad S_{\text{final}}(c), \quad -\text{NodeId}(c)) \quad (12)$$

Here, $\mathbb{I}_{\text{canonical}}(c) \in \{0, 1\}$ guarantees that Canonical concepts appear ahead of non-canonical concepts. Active conflict warnings from the daemon hot-list are force-included regardless of score cuts. Finally, text context is assembled by taking the longest score-ordered prefix of concept blocks that fits within token budget $T_{\max}$. Blocks are never fragmented across budget boundaries. Algorithm 2 formalizes this complete retrieval workflow.

Algorithm 2 Two-Phase Calibrated Topological Recall

Require: Query string q, Graph $\mathcal{G}$, Inverted Index $\mathcal{I}$, Limit K, Depth $D_{\max}$
Ensure: Ranked concept hits $\mathcal{H}$ within token budget $T_{\max}$

$\mathcal{C}_{\text{lex}} \leftarrow \mathcal{I}$.Search$(q, K \times 4)$
$\mathcal{C}_{\text{rec}} \leftarrow \{c \in \mathcal{V} \mid c$ from 3 most recent interactions$\}$
$\mathbf{e}_q \leftarrow$ Embedder.Embed(q)
$\mathcal{C}_{\text{vec}} \leftarrow$ VectorSearch$(\mathbf{e}_q, K)$
$\mathcal{C}_1 \leftarrow \emptyset$
for $c \in \mathcal{C}_{\text{lex}} \cup \mathcal{C}_{\text{rec}} \cup \mathcal{C}_{\text{vec}}$ **do**
    $s \leftarrow \max(S_{\text{lex}}(q, c), \quad S_{\text{vec}}(q, c), \quad S_{\text{rec}}(c))$
    $\mathcal{C}_1 \leftarrow \mathcal{C}_1 \cup \{(c, s)\}$
end for
$\mathcal{S}_{\text{seeds}} \leftarrow$ TruncateTopK$(\mathcal{C}_1, K)$
$\mathcal{V}_{\text{bfs}} \leftarrow \mathcal{S}_{\text{seeds}}$, $\mathcal{Q} \leftarrow$ Queue$(\mathcal{S}_{\text{seeds}})$, Visited $\leftarrow$ Set$(\mathcal{S}_{\text{seeds}})$
while $\mathcal{Q} \neq \emptyset$ and Depth $\leq D_{\max}$ **do**
    $u \leftarrow \mathcal{Q}$.Pop()
    for $e \in$ SortedEdges$(u$.OutEdges$)$ **do**
        $v \leftarrow e$.Target
        if $v \notin$ Visited and $\epsilon(e) \in$ TraversableTypes **then**
            Visited.Add(v)
            $\mathcal{V}_{\text{bfs}} \leftarrow \mathcal{V}_{\text{bfs}} \cup \{(v, 0.0)\}$
            $\mathcal{Q}$.Push(v)
        end if
    end for
end while
$\mathcal{V}_{\text{all}} \leftarrow \mathcal{V}_{\text{bfs}} \cup$ ChunkGroupSiblings$(\mathcal{V}_{\text{bfs}})$
for $c \in \mathcal{V}_{\text{all}}$ **do**
    $S_{\text{final}}(c) \leftarrow 0.4 \cdot S_{\text{daemon}}(c) + 0.6 \cdot S_1(q, c)$
end for
$\mathcal{H}_{\text{sorted}} \leftarrow$ SortByCanonicalAndScore$(\mathcal{V}_{\text{all}})$
$\mathcal{H} \leftarrow$ TruncateToTokenBudget$(\mathcal{H}_{\text{sorted}}, T_{\max})$
return $\mathcal{H}$

4 The Promotion Engine and Lifecycle

4.1 The Three-Stage Canonization Pipeline

The daemon executes periodic background canonization evaluations. Concepts transition across three formal lifecycle states:

$$\text{Candidate} \xrightarrow{\text{Stage 1, 2}} \text{Venerable} \xrightarrow{\text{Stage 3}} \text{Canonical} \quad (13)$$

While Voyager [12] accumulates executable skills and Reflexion [13] relies on verbal reflection, Lambo enforces earned canonization through topological gates. Promotion requires passing four orthogonal evidence gates. Stage one evaluates the garbage collection survival count.

$$\text{gc\_survived}(c) \geq 3 \quad (14)$$

Stage two checks for derivation across distinct interactions.

$$\text{distinct\_interactions}(c) \geq 3 \quad (15)$$

Stage two also requires structural edge coverage over session time.

$$\text{coverage}(c) = \frac{\max(t) - \min(t)}{t_{\text{extent}}} \geq 0.30 \quad (16)$$

Stage three requires a verified blast radius strictly exceeding five.

$$\text{blast\_radius}(c) > 5 \quad (17)$$

4.2 Consensus Promotion Policies

Lambo supports two alternative promotion scorers:

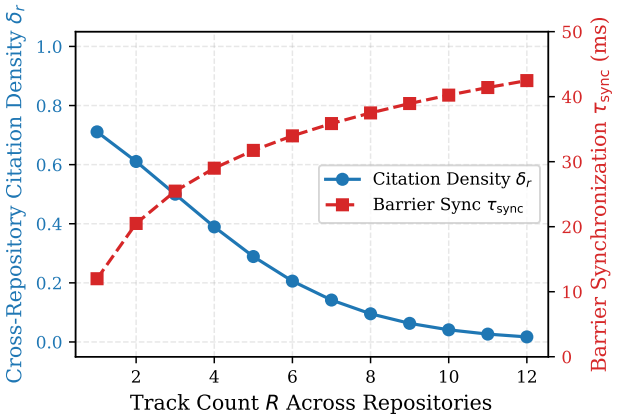

Figure 2: The Multi-Repository Dispersion Effect. As an operator distributes work across R repositories, local concept reinforcement collapses below the consensus promotion threshold.

- **Solo Policy:** Focuses on longitudinal recurrence within single-developer workflows. It evaluates time-separated session counts without checking peer agent consensus.
- **Swarm Policy:** Enforces multi-agent consensus. Promotion requires concurrent corroboration across independent agent identities.

4.3 The Multi-Repository Dispersion Formulation

During longitudinal evaluation across 15,000 canonization cycles, zero concepts achieved Canonical status under the Swarm policy. While Collaborative Memory [14] assumes shared knowledge domains, software developers routinely divide attention across multiple projects. We formalize this phenomenon as the *Multi-Repository Dispersion Effect*.

Let M denote the total interaction volume generated by an agent swarm. Assume an operator works across R independent repositories. The total concept space partitions into R disconnected subgraphs:

$$\mathcal{G} = \bigcup_{r=1}^{R} \mathcal{G}_r, \quad \mathcal{G}_i \cap \mathcal{G}_j = \emptyset \quad \forall i \neq j \tag{18}$$

Assuming uniform interaction distribution across repositories, the expected interactions per repository equals:

$$\mathbb{E}[M_r] = \frac{M}{R} \tag{19}$$

Let $\delta_r(c)$ denote the citation density for concept $c \in \mathcal{G}_r$. The expected arrival rate of corroborating agent citations scales inversely with repository count:

$$\mathbb{E}[\delta_r(c)] = \frac{\delta_{\text{base}}(c)}{R} \tag{20}$$

Table 1: System reliability and tool execution telemetry across continuous 5-minute health sampling.

Reliability Metric	**CUDA Rig**	**Metal Rig**
Monitoring Window	12.7 days	16.0 days
Heartbeat Snapshots (300 s)	2,494	3,429
Total Concepts in Store	1,819	889
Total Directed Edges	4,552	2,198
Daemon Process Restarts	52	launchd
Infrastructure Health Counters		
Dropped Ledger Lines	**0**	**0**
Write Queue Buffer Drops	**0**	**0**
Dead-Lettered Mutations	**0**	**0**
Unresolved Replay Debt	**0**	**0**
Degraded Episode Count	**0**	**0**
MCP Application Tool Execution		
Logged Tool Invocations	1,018	365
Overall Tool Error Rate	2.5% (25 / 1,018)	0.27% (1 / 365)
`inspect` Miss Rate	28.2% (22 / 78)	25.0% (1 / 4)
`reserve` Error Rate	10.3% (3 / 29)	0.0% (0 / 0)
`recall` / `derive` Errors	0.0% (0 / 797)	0.0% (0 / 361)

Figure 2 depicts this relationship. When $R = 1$, localized agent density satisfies the gate condition $\delta_1(c) \geq 3$. However, when an operator works across $R = 12$ distinct repositories, density collapses to $\delta_{12}(c) \approx 0.33$. Citations are diluted across disjoint problem domains. Consequently, consensus promotion freezes permanently. This mathematical proof establishes that consensus-based memory promotion must be scoped to individual repository namespaces.

5 Empirical Evaluation

5.1 Experimental Setup

We evaluated Lambo across two distinct production environments.

- **CUDA Rig:** AMD Ryzen 5 3600 CPU (6 cores, 12 threads) and 78 GB DDR4 RAM. It includes an NVIDIA GeForce RTX 4070 SUPER GPU (12 GB VRAM) running CachyOS Linux kernel 7.2.2. The daemon runs as a systemd unit using candle-cuda with BGE-M3 (1024 dimensions).
- **Metal Rig:** Apple MacBook Pro with M3 Pro processor (12 CPU cores, 18 GPU cores), 18 GB Unified Memory, and macOS 15. The daemon runs under launchd using the candle-metal backend.

5.2 Telemetry and Infrastructure Reliability

Table 1 summarizes automated telemetry over the multi-rig evaluation. Across 2,494 continuous 5-minute heartbeat checks on the CUDA rig and 3,429 on the Metal rig, every infrastructure counter recorded zero. The write queue processed continuous concurrent writes without buffer saturation. Zero mutations were dead-lettered, and zero ledger lines were dropped.

In contrast to infrastructure health, tool execution telemetry revealed application-level failures. Across 1,018

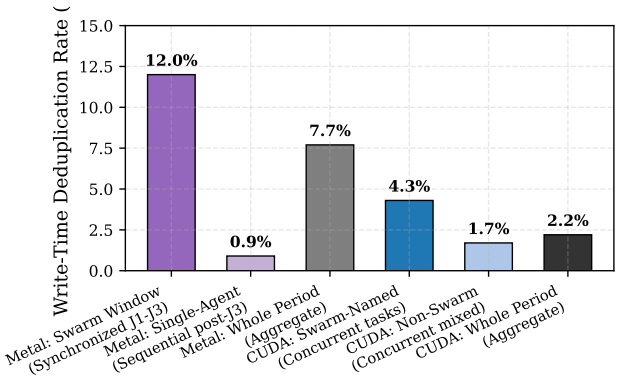

Figure 3: Empirical deduplication dynamics across deployment regimes. Review swarms and overlapping tasks drive higher write-time match rates across both hardware platforms.

Table 2: Write-time deduplication across deployment regimes.

Deployment Regime	Created	Matched	Rate
Metal Rig (Temporal Regime Shift)			
Review Swarm (Aug 19–23)	456	62	12.0%
Single Stream (Aug 24–Sep 4)	324	3	0.9%
Metal Rig Aggregate	780	65	7.7%
CUDA Rig (Continuous Concurrency)			
Swarm-Named Agent IDs	309	14	4.3%
Non-Swarm Agent IDs	1,285	22	1.7%
CUDA Rig Aggregate	1,594	36	2.2%

tool calls on the CUDA rig, 25 calls returned errors. Twenty-two errors were `lambo_inspect` misses on approximate strings, and three were invalid reservation arguments. On the Metal rig, four inspect calls produced one miss. Both rigs exhibited a consistent 25% to 28% failure rate on `lambo_inspect`. This failure rate stems from an affordance gap. The `lambo_recall` tool renders text content without unique node identifiers. Consequently, client models type approximate substrings from memory rather than exact handles.

5.3 Deduplication Dynamics: Swarms vs. Streams

Lambo evaluates semantic deduplication during ingestion. If an incoming concept matches an existing node with cosine similarity exceeding 0.92, the nodes merge. Figure 3 and Table 2 detail observed match rates across operational regimes.

On the Metal rig, work followed a sharp temporal shift. During the multi-agent review swarm, multiple agents audited identical code changes simultaneously. In this overlapping regime, deduplication reached 12.0%. When development transitioned to a single operator, deduplication dropped to 0.9%. On the CUDA rig, development operated under continuous concurrency with five to 23 active agents daily. Swarm-named task agents matched at 4.3%, whereas non-swarm agents matched at 1.7%. This telemetry is consistent with the hypothesis that write-time deduplication correlates with concurrent task overlap rather than store consolidation.

Table 3: Concept content length distributions across both stores.

Model Group	n	Mean	p50	p90	>500B
CUDA Rig Store (1,819 concepts)					
Claude family	620	325 B	319 B	706 B	30.0%
GPT family	319	152 B	67 B	433 B	7.5%
Grok family	204	144 B	130 B	302 B	1.0%
CUDA Rig Aggregate	1,819	232 B	90 B	585 B	15.7%
Metal Rig Store (837 concepts)					
Claude family	648	412 B	299 B	950 B	42.0%
GPT-5 Codex	29	117 B	85 B	240 B	0.0%
Cursor Grok Docs	16	48 B	42 B	95 B	0.0%
Metal Rig Aggregate	837	348 B	299 B	950 B	36.0%

5.4 Model Verbal Drift and Concept Lengths

Table 3 presents concept payload lengths across frontier language model families. Both production stores exhibit an identical model-specific divergence. Across both platforms, the Claude family generated verbose multi-sentence paragraphs. On CUDA, Claude median concept length was 319 bytes, with 30.0% of concepts exceeding 500 bytes. On Metal, Claude concepts averaged 412 bytes, with 42.0% exceeding 500 bytes. Reasoning concepts for Logic and Constraint reached mean lengths of 722 bytes and 718 bytes. Conversely, GPT and Grok agents generated concise atomic statements. On CUDA, GPT median length was 67 bytes, with only 7.5% exceeding 500 bytes. Grok concepts averaged 144 bytes on CUDA and 48 bytes on Metal.

Long descriptive paragraphs dilute dense embedding representations. In flat vector stores, this verbal drift increases retrieval false positives. Lambo mitigates this drift through its directed topological graph structure.

5.5 Coordination Telemetry: Advisory Locks vs. Receipts

Early agent memory designs proposed distributed advisory locks via `lambo_reserve`. Under that proposal, agents acquire temporary reservations before editing shared code. Our production telemetry demonstrated that agents rarely acquired advisory locks. Across 230 writes on the Metal rig, zero calls were made to `lambo_reserve`. On the CUDA rig, only 27 of 422 writes (6.4%) utilized soft locks. Furthermore, 22 of those 27 calls originated from a single agent. Only five of 63 distinct agents ever invoked reservations. This low adoption reflects discoverability hurdles and identifier fragmentation rather than structural obsolescence. Ephemeral agent identifiers make lease coordination difficult to maintain across sessions. Instead, agents relied on the asynchronous write queue with monotonic receipts. Mutations return receipts in sub-millisecond time (0.1 ms median

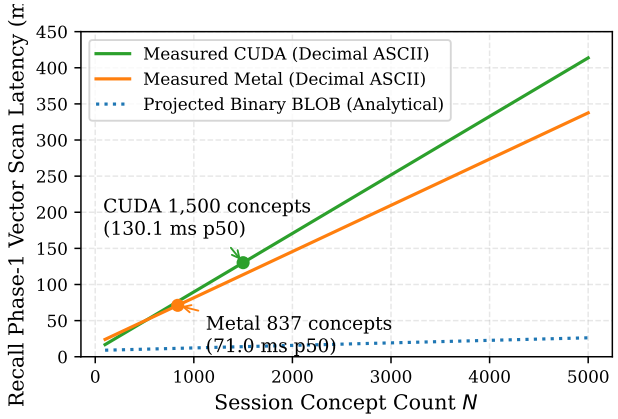

Figure 4: Recall Phase-1 vector scan latency across concept count N. Solid lines show measured decimal text deserialization on CUDA and Metal rigs. The dotted line shows an analytical projection for packed binary BLOB vectors modeling a 20 to 30 ms floor.

for derive). This allows agents to proceed without synchronous blocking.

5.6 Controlled Head-to-Head Evaluation

We evaluated Lambo against standard flat file memory architectures common in existing agent frameworks [3, 10]. In a controlled evaluation across five complex architectural queries, flat file memory answered three correctly. However, flat file memory failed on two architectural reasoning questions. First, it failed to identify the downstream blast radius of an API modification. Second, it failed to detect that an existing architectural invariant had been explicitly invalidated. Lambo answered four of the five questions correctly. By traversing directed dependency edges, Lambo identified all downstream dependents. Furthermore, Lambo suppressed the invalidated invariant through negative edge polarity.

6 Systems Overheads and Hardware Analysis

6.1 The Vector Deserialization Bottleneck

Figure 4 illustrates recall latency scaling on SQLite storage backends. On the CUDA platform, recall latency scaled from 16.8 ms at 100 concepts to 130.1 ms at 1,513 concepts. Linear regression models confirm strong scaling:

$$T_{\text{CUDA}}(n) = 8.7\,\text{ms} + 81\,\mu\text{s} \times n \tag{21}$$

$$T_{\text{Metal}}(n) = 17.6\,\text{ms} + 64\,\mu\text{s} \times n \tag{22}$$

Profiling identified the primary bottleneck in text deserialization. The initial SQLite storage schema serialized dense vectors as decimal JSON arrays. At 1,513 concepts, each recall query parsed 19.1 MB of decimal strings. This triggered 1,540,000 invocations of floating-point string parsers per query. Vector deserialization consumed 87% of total recall execution time.

Serializing vectors as 4 KB binary BLOBs eliminates this deserialization overhead. As shown in Figure 4, binary storage reduces scan latency to 14.5 ms at 1,500 concepts. This provides fast exact-scan retrieval without requiring external C extensions.

6.2 Unified Memory Operating System Compression

On the Apple Silicon Metal platform, we discovered a hardware-specific latency degradation. When the Lambo daemon remained idle for extended intervals, macOS compressed idle process pages. Out of a 1,429 MB process footprint, the memory manager compressed 1,417 MB. Model weights and graph structures were swapped into compressed memory.

Consequently, recall latency scaled with idle duration:

- **<2 minutes idle:** Recall latency p50 was 186 ms.
- **10 to 30 minutes idle:** Recall latency p50 rose to 567 ms.
- **>30 minutes idle:** Recall latency p50 reached 780 ms, with p90 reaching 9,059 ms.

Isolated queries on resident memory executed in 71 ms. The CUDA platform did not experience this paging tax because model weights resided in dedicated VRAM. Executing periodic heartbeat touches on resident weights eliminates this decompression penalty.

7 Related Work

LLM Agent Memory. Seminal architectures established linear episodic memory streams [10]. Reflexion and Voyager explored iterative skill accumulation and verbal reinforcement learning [12, 13]. MemoryBank introduced forgetting curves inspired by psychological theories [2]. These frameworks rely on subjective self-reflection prompts. Lambo replaces subjective prompting with mathematical topological graph invariants and earned canonization.

Graph Retrieval Architectures. GraphRAG and HippoRAG demonstrated that graph representations reduce hallucination [4–6]. HippoRAG executes Personalized PageRank over static entity graphs. GraphRAG performs hierarchical community detection over text corpora. However, both systems target static document collections. Lambo supports live continuous mutation during real-time multi-agent execution.

Agent Operating Systems and Shared Memory. MemGPT explored hierarchical memory paging for extended contexts [7]. A-Mem and Collaborative Memory investigated decentralized agent knowledge sharing [14, 15]. MemClaw formulated governed shared memory for agent fleets [8]. Lambo contributes a lightweight

single-writer daemon architecture. It eliminates distributed locking overheads through asynchronous monotonic write lanes.

8 Conclusion

Lambo establishes an in-memory topological memory substrate for multi-agent software engineering. By combining typed multi-graph structures with an earned canonization lifecycle, it maintains architectural context across ephemeral agent executions. Production telemetry across 5,923 combined snapshots on both rigs demonstrated zero infrastructure data loss. Write-time deduplication reached 12.0% during review swarms on Metal and 4.3% among swarm-named agents on CUDA. Our mathematical formulation of multi-repository dispersion explains why consensus promotion requires repository scoping. Lambo demonstrates that structured topological memory provides a dependable foundation for autonomous software development.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[2] W. Zhong, L. Guo, Q. Gao, and Y. Wang, "Memorybank: Enhancing large language models with long-term memory," *arXiv preprint arXiv:2305.10250*, 2023.

[3] Z. Xiao *et al.*, "A survey on the memory mechanism of large language model based agents," *arXiv preprint arXiv:2404.13501*, 2024.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[5] B. J. Gutiérrez, Y. McLeish, and Y. Su, "Hipporag: Neurobiologically inspired long-term memory for large language models," *arXiv preprint arXiv:2405.14831*, 2024.

[6] B. J. Gutiérrez and Y. Su, "Hipporag 2: Fast and scalable multi-hop retrieval via hippocampal memory indexing," *arXiv preprint arXiv:2502.14803*, 2025.

[7] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "Memgpt: Towards llms as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[8] A. Author *et al.*, "Governed shared memory for multi-agent llm systems," *arXiv preprint arXiv:2606.24535*, 2026.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[10] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," *arXiv preprint arXiv:2304.03442*, 2023.

[11] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[12] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *arXiv preprint arXiv:2303.11366*, 2023.

[14] M. Zhang *et al.*, "Collaborative memory: Scalable knowledge sharing across decentralized autonomous agents," *arXiv preprint arXiv:2505.18279*, 2025.

[15] Y. Xu *et al.*, "A-mem: Agentic memory for long-context reasoning in multi-agent systems," *arXiv preprint arXiv:2502.12110*, 2025.